

Cheat Sheet

Wednesday, March 12, 2025 11:33 PM

Week 2	Const	C array	Memory Models
String operations	- immutable	- stack	
insert()	- means we cannot change	char s[6];	o local variables
erase()	Functions	s[0] = 'h'	o allocation = automatic
resize()	- vector to a function	s[1] = 'e'	o deallocation = when function exists
substr()	- pass by value vector<int> vec		- static
contains()	o copied		o alloc= at compile
ends_with()	- pass by reference vector<int> &vec		o dealloc = when program terminates
starts_with()	o no copy		- heap
Numeric Types	- pass by const reference const vector<int> &vec		o allocate = int *p = new int (42);
	o no copy and cannot be changed in the func		o deallocate = delete p;
		C++ pointers	
		- gets address of something	
		- dereference address to get to what it's pointing to	
		o *p = &x	
		o int a=0; int *p = &a; (*p)++; int x = a;	
		o dereference above makes use increment a	

Week 1
C++ class based
A class is treated like a Type
Object is instance of a class

Week 4

STL

- containers
 - o sequence
 - vector
 - sequence
 - dynamic size
 - add/remove anywhere
 - bounds safety
 - list
 - deque
 - arrays
 - forward_list - linked list
 - o adaptors
 - queue, priority queue,
 - stack
 - LIFO
 - encapsulated object of vector or deque
 - o associative - O(logn)
 - set - balance binary tree
 - multiset - bst
 - map - bst
 - multimap - bst
 - o unordered associative
 - unordered set, multiset, map, multimap - hashmap
- iterators
 - o point at begin and end move forward backward
 - o right place - front end
 - o direction - forward backward
 - o iterator
 - o reverse iterator
 - o can't insert at .end()
 - o back_inserter() could work
- algorithms
 - o auto- automatically finds type

Week 3	Casting	Exceptions (try, catch)
spiral rule	- static_cast	- exception must be value
	o conversions safe for compiler	o int, float, char
	o between related types	o string
	o static_cast<new_type>(expression)	o any object
	o ex int to double	o pointer to something
	- dynamic_cast	- report exceptions
	o downcasting	o throw <expr> errr
	o proolly don't use	▪ a statement
	- const_cast	▪ value of expression thrown
	o removing const	- handle exceptions
	o only use if underlying data mutable	o catch
	o const int a = 10;	
	o int* modifiable = const_cast<int*>(&a);	
References	- reinterpret_cast	
- only exist c++	o convert the pointer to any other type of pointer	
- serves same purpose as pointers	o reinterpret_cast<new_type>(expression)	
- helps with copying saving data space		
	Control Flow c++	
	- try-catch-throw	
	- range loop	
	- intervals	

Week 6

Inheritance

- promote code reuse
- theory
 - o support type refinements
 - o subtyping through subclassing
- $A <: B$ = A is subtype of B
 - o B is expected you can provide A

Inheritance and privacy

- can alter privacy of what you want to inherit
 - o public
 - public - public
 - protected - protected
 - private - private
 - when overriden use above setting
 - when A inherits B everyone knows
 - o protects
 - public - protected
 - protected - private
 - when override use above privacy setting
 - when A inherits B only A and subclasses know
 - o private
 - public - private
 - protected - private
 - privacy setting using when override called
 - only A and A only knows it inherits from B
 - o class Human: public Mammal - inheritance
 - o when inheritance upgrades/refines methods it inherits that means overriding
 - o dynamic binding - polymorphism
 - o

Polymorphism - object behave different based of type/class

- compile-time
 - o function overloading
 - o operator overloading
- runtime poly
 - o ability for object to respond based of dynamic type rather than compile time type
 - o function overriding and virtual function
 - o virtual switches from static to dynamic

- ex: runtime

```
class Mammal {
public:
    Mammal() {}
    virtual void print() {
        std::cout << "I (" << this << ")'m a mammal!" << std::endl;
    }
};
```

```
class Human : public Mammal {
public:
    Human() {}
    virtual void print() { //if no virtual here still treated like it has it
        std::cout << "I (" << this << ")'m a human!" << std::endl;
    }
};
```

Final

- cannot be override after override final, this would be the last override
- class B { public: virtual void f(int) {std::cout << "B::f" << std::endl;}}; class D : public B { public: virtual void f(int) override final {std::cout << "D::f" << std::endl;}}

Deallocation

- make destructor polymorphic
- virtual ~Base
- must also delete poly borrower ~Derived

Multiple Inheritance Diamond

- parent classes will inherit virtual
- calls Student: virtual public person
- this would avoid confusion with the compile where the class pulls shared methods